

```

import torch, os, shutil

# ----- Optional: load checkpoint from a persistent dataset -----
# If you created a dataset named "crack-checkpoints" (see instructions
# at the bottom),
# attach it to this notebook, then this cell will restore your
# previous state.
CKPT_DATASET = '/kaggle/input/crack-checkpoints'    # <-- change if
your dataset name differs
LOCAL_CKPT = '/kaggle/working/checkpoint.pth'
LOCAL_BEST = '/kaggle/working/best_model.pth'

if os.path.exists(f'{CKPT_DATASET}/checkpoint.pth'):
    shutil.copy(f'{CKPT_DATASET}/checkpoint.pth', LOCAL_CKPT)
    shutil.copy(f'{CKPT_DATASET}/best_model.pth', LOCAL_BEST)
    print('☐ Checkpoint loaded from persistent dataset')
else:
    print('❗ No persistent checkpoint – using local one if exists')

# ----- Quick GPU check -----
print("☐ GPU STATUS")
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))

❗ No persistent checkpoint – using local one if exists
☐ GPU STATUS
CUDA available: True
GPU name: Tesla T4

import os, cv2, numpy as np, torch, torch.nn as nn,
torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from glob import glob
import random, time

# ----- SETTINGS -----
RESET_TRAINING = False                # Keep False to resume from
checkpoint
WORK_DIR = '/kaggle/working'
CKPT_PATH = os.path.join(WORK_DIR, 'checkpoint.pth')
BEST_PATH = os.path.join(WORK_DIR, 'best_model.pth')

if RESET_TRAINING:
    for f in [CKPT_PATH, BEST_PATH]:
        if os.path.exists(f): os.remove(f)
    print('Forced reset - training from scratch')
else:
    if os.path.exists(CKPT_PATH):
        print('Local checkpoint found - resuming')

```

```

else:
    print('No checkpoint - starting from scratch')

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.backends.cudnn.benchmark = True
SEED = 42
torch.manual_seed(SEED); np.random.seed(SEED); random.seed(SEED)

# ===== GroupNorm helper =====
def norm_layer(channels, groups=8):
    return nn.GroupNorm(min(groups, channels), channels)

# ===== Dataset (fast) =====
class CrackDataset(Dataset):
    def __init__(self, img_dir, mask_dir, size=(320,320),
augment=False):
        self.img_paths = sorted(glob(os.path.join(img_dir, '*')))
        self.mask_dir = mask_dir
        self.size = size
        self.augment = augment

    def __len__(self): return len(self.img_paths)

    def __getitem__(self, idx):
        img_path = self.img_paths[idx]
        name = os.path.basename(img_path)
        mask_path = os.path.join(self.mask_dir, name)

        img = cv2.imread(img_path)
        if img is None: raise FileNotFoundError(img_path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        mask = cv2.imread(mask_path, 0)
        if mask is None: raise FileNotFoundError(mask_path)

        img = cv2.resize(img, self.size,
interpolation=cv2.INTER_LINEAR)
        mask = cv2.resize(mask, self.size,
interpolation=cv2.INTER_NEAREST)

        if self.augment:
            if random.random() < 0.5:
                img = cv2.flip(img, 1)
                mask = cv2.flip(mask, 1)
            if random.random() < 0.5:
                img = cv2.flip(img, 0)
                mask = cv2.flip(mask, 0)
            alpha = 0.8 + 0.4 * random.random()
            beta = int(20 * random.random() - 10)
            img = cv2.convertScaleAbs(img, alpha=alpha, beta=beta)

```

```

        img = torch.from_numpy(img).permute(2,0,1).float() / 255.0
        mask = (torch.from_numpy(mask).unsqueeze(0).float() / 255.0) >
0.5
        return img, mask.float()

# ===== Novel Modules (unchanged) =====
class AOSC(nn.Module):
    def __init__(self, ch):
        super().__init__()
        self.h = nn.Conv2d(ch, ch, (1,9), padding=(0,4), groups=ch)
        self.v = nn.Conv2d(ch, ch, (9,1), padding=(4,0), groups=ch)
        self.fuse = nn.Conv2d(ch*2, ch, 1)
        self.att = nn.Sequential(nn.AdaptiveAvgPool2d(1),
nn.Conv2d(ch, ch//4, 1),
nn.ReLU(), nn.Conv2d(ch//4, ch, 1),
nn.Sigmoid())
    def forward(self, x):
        h,v = self.h(x), self.v(x)
        fused = self.fuse(torch.cat([h,v],1))
        return fused * self.att(fused)

class GatedRCP(nn.Module):
    def __init__(self, ch):
        super().__init__()
        self.conv = nn.Conv2d(ch, ch, 3, padding=1)
    def forward(self, x):
        left = torch.roll(x, 1, dims=3)
        right = torch.roll(x, -1, dims=3)
        gate = torch.sigmoid(self.conv(x))
        return x + gate * (self.conv(left) + self.conv(right))

class SOTS(nn.Module):
    def __init__(self, ch):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(ch, ch//4), nn.ReLU(),
nn.Linear(ch//4, ch), nn.Sigmoid())
    def forward(self, x):
        w = torch.mean(x.view(x.size(0), x.size(1), -1), 2)
        return x * self.fc(w).view(-1, x.size(1), 1, 1)

class MultiScaleFusion(nn.Module):
    def __init__(self, ch_list, out_ch):
        super().__init__()
        self.pre_conv = nn.ModuleList([nn.Conv2d(c, out_ch, 1) for c
in ch_list])
        self.out = nn.Conv2d(out_ch * len(ch_list), out_ch, 3,
padding=1)
    def forward(self, features):
        target_size = features[-1].shape[2:]
        aligned = []

```

```

        for conv, f in zip(self.pre_conv, features):
            f = conv(f)
            if f.shape[2:] != target_size:
                f = F.interpolate(f, size=target_size,
mode='bilinear', align_corners=False)
            aligned.append(f)
        return self.out(torch.cat(aligned, dim=1))

# ===== CrackProMax (320 input) =====
class CrackProMax(nn.Module):
    def __init__(self, input_size=320):
        super().__init__()
        self.input_size = input_size

        self.stem = nn.Sequential(
            nn.Conv2d(3, 32, 7, stride=2, padding=3, bias=False),
            norm_layer(32), nn.ReLU(inplace=True))

        self.enc1 = nn.Sequential(
            AOSC(32),
            nn.Conv2d(32, 64, 3, padding=1), norm_layer(64),
nn.ReLU(inplace=True),
            nn.MaxPool2d(2))

        self.enc2 = nn.Sequential(
            nn.Conv2d(64, 128, 3, padding=1), norm_layer(128),
nn.ReLU(inplace=True),
            nn.MaxPool2d(2))

        self.enc3 = nn.Sequential(
            nn.Conv2d(128, 256, 3, padding=1), norm_layer(256),
nn.ReLU(inplace=True),
            nn.MaxPool2d(2))

        self.enc4 = nn.Sequential(
            nn.Conv2d(256, 512, 3, padding=1), norm_layer(512),
nn.ReLU(inplace=True),
            nn.MaxPool2d(2))

        self.bottleneck = nn.Sequential(
            nn.Conv2d(512, 512, 3, padding=2, dilation=2, groups=512,
bias=False),
            nn.Conv2d(512, 512, 1), norm_layer(512),
nn.ReLU(inplace=True),
            SOTS(512))

        self.up4 = nn.ConvTranspose2d(512, 256, 2, stride=2)
        self.rcp4 = GatedRCP(256)
        self.skip4 = nn.Conv2d(256+256, 256, 1)

```

```

self.up3 = nn.ConvTranspose2d(256, 128, 2, stride=2)
self.rcp3 = GatedRCP(128)
self.skip3 = nn.Conv2d(128+128, 128, 1)

self.up2 = nn.ConvTranspose2d(128, 64, 2, stride=2)
self.rcp2 = GatedRCP(64)
self.skip2 = nn.Conv2d(64+64, 64, 1)

self.up1 = nn.ConvTranspose2d(64, 32, 2, stride=2)
self.rcp1 = GatedRCP(32)
self.skip1 = nn.Conv2d(32+32, 32, 1)

self.ms_fusion = MultiScaleFusion([256, 128, 64, 32], 32)

self.mask_head = nn.Conv2d(32, 1, 1)
self.edge_head = nn.Conv2d(32, 1, 1)

self.ds_head3 = nn.Conv2d(128, 1, 1)
self.ds_head2 = nn.Conv2d(64, 1, 1)
def forward(self, x, return_aux=False):
    s0 = self.stem(x)
    e1 = self.enc1(s0)
    e2 = self.enc2(e1)
    e3 = self.enc3(e2)
    e4 = self.enc4(e3)
    b = self.bottleneck(e4)

    d4 = self.up4(b)
    d4 = self.rcp4(d4)
    d4 = self.skip4(torch.cat([d4, e3], 1))

    d3 = self.up3(d4)
    d3 = self.rcp3(d3)
    d3 = self.skip3(torch.cat([d3, e2], 1))

    d2 = self.up2(d3)
    d2 = self.rcp2(d2)
    d2 = self.skip2(torch.cat([d2, e1], 1))

    d1 = self.up1(d2)
    d1 = self.rcp1(d1)
    d1 = self.skip1(torch.cat([d1, s0], 1))

    fused = self.ms_fusion([d4, d3, d2, d1])

    mask_logits = self.mask_head(fused)
    edge_logits = self.edge_head(fused)

    if mask_logits.shape[2] != self.input_size:

```

```

        mask_logits = F.interpolate(mask_logits,
size=(self.input_size, self.input_size),
                                     mode='bilinear',
align_corners=False)
        edge_logits = F.interpolate(edge_logits,
size=(self.input_size, self.input_size),
                                     mode='bilinear',
align_corners=False)

        if return_aux:
            ds3 = self.ds_head3(d3)
            ds2 = self.ds_head2(d2)
            ds3 = F.interpolate(ds3, size=(self.input_size,
self.input_size), mode='bilinear', align_corners=False)
            ds2 = F.interpolate(ds2, size=(self.input_size,
self.input_size), mode='bilinear', align_corners=False)
            return mask_logits, edge_logits, ds3, ds2
        else:
            return mask_logits, edge_logits

# ===== Losses (cLDice COMMENTED OUT) =====
def create_edge_target(mask):
    with torch.no_grad():
        sobel_x = torch.tensor([[[-1,0,1],[-2,0,2],[-1,0,1]],
device=mask.device, dtype=torch.float32).view(1,1,3,3)
        sobel_y = torch.tensor([[[-1,-2,-1],[0,0,0],[1,2,1]],
device=mask.device, dtype=torch.float32).view(1,1,3,3)
        edge = (F.conv2d(mask, sobel_x, padding=1).abs() +
F.conv2d(mask, sobel_y, padding=1).abs()).clamp(0,1)
    return edge

def focal_loss(logits, target, alpha=0.25, gamma=2.0,
pos_weight=10.0):
    prob = torch.sigmoid(logits).clamp(min=1e-7, max=1-1e-7)
    pt = prob*target + (1-prob)*(1-target)
    weight = alpha*target + (1-alpha)*(1-target)
    weight_pos = pos_weight*target + (1-target)
    loss = -weight_pos * weight * ((1-pt).pow(gamma)) * torch.log(pt)
    return loss.mean()

def dice_loss(prob, target):
    smooth = 1e-5
    num = 2*(prob*target).sum() + smooth
    den = prob.sum() + target.sum() + smooth
    return 1 - num/den

def loss_fn(mask_logits, edge_logits, mask_gt, epoch=0,
ds_logits=None, ds_weight=0.1):
    prob = torch.sigmoid(mask_logits).clamp(min=1e-7, max=1-1e-7)
    mask_loss = 0.7*focal_loss(mask_logits, mask_gt, pos_weight=15.0)

```

```

+ \
    0.3*dice_loss(prob, mask_gt)
mask_loss = mask_loss * (1 + 2 * mask_gt.mean().clamp(min=0))

edge_gt = create_edge_target(mask_gt)
edge_loss = F.binary_cross_entropy_with_logits(edge_logits,
edge_gt)
total = mask_loss + 0.35*edge_loss

if ds_logits is not None:
    ds3_logits, ds2_logits = ds_logits
    if ds3_logits is not None:
        ds3_prob = torch.sigmoid(ds3_logits).clamp(min=1e-7,
max=1-1e-7)
        total += ds_weight * (0.5*focal_loss(ds3_logits, mask_gt,
pos_weight=10) +
                                0.5*dice_loss(ds3_prob, mask_gt))
    if ds2_logits is not None:
        ds2_prob = torch.sigmoid(ds2_logits).clamp(min=1e-7,
max=1-1e-7)
        total += ds_weight * (0.5*focal_loss(ds2_logits, mask_gt,
pos_weight=10) +
                                0.5*dice_loss(ds2_prob, mask_gt))

# --- CLDICE IS COMPLETELY REMOVED TO STOP GPU CRASH ---
# (uncomment if you later want it back)
# if epoch > 30 and epoch % 3 == 0:
#     cl_loss = compute_cLDice(prob.detach(), mask_gt)
#     total = total + 0.1 * cl_loss

return total

# ===== Training Loop =====
def train_model(train_loader, val_loader, model, epochs=100,
patience=50):
    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-6,
weight_decay=1e-4)
    warmup_epochs, target_lr = 5, 5e-4
    scheduler = torch.optim.lr_scheduler.ReduceLRonPlateau(optimizer,
mode='max', factor=0.5, patience=5)
    scaler = torch.amp.GradScaler('cuda')

    start_epoch, best_dice, counter = 0, 0.0, 0
    last_val_dice = 0.0

    if os.path.exists(CKPT_PATH) and not RESET_TRAINING:
        print('Resuming from checkpoint...')
        try:
            ckpt = torch.load(CKPT_PATH, map_location=DEVICE,
weights_only=False)

```

```

        model.load_state_dict(ckpt['model_state'])
        optimizer.load_state_dict(ckpt['optimizer_state'])
        scheduler.load_state_dict(ckpt['scheduler_state'])
        if 'scaler_state' in ckpt:
            scaler.load_state_dict(ckpt['scaler_state'])
            start_epoch = ckpt['epoch'] + 1
            best_dice = ckpt.get('best_dice', 0.0)
            counter = ckpt.get('patience_counter', 0)
            print(f'Resumed epoch {start_epoch}, best Dice:
{best_dice:.4f}')
        except Exception as e:
            print(f'Failed to resume ({e}), starting from scratch.')
            os.remove(CKPT_PATH)

    for epoch in range(start_epoch, epochs):
        # Warm-up
        if epoch < warmup_epochs and start_epoch == 0:
            lr = target_lr * (epoch+1) / warmup_epochs
            for pg in optimizer.param_groups: pg['lr'] = lr
            print(f'Warm-up epoch {epoch+1}: LR={lr:.2e}')
        elif epoch == warmup_epochs and start_epoch == 0:
            for pg in optimizer.param_groups: pg['lr'] = target_lr
            print(f'Warm-up done - LR={target_lr:.2e}')

        model.train(); train_loss = 0.0; nan_count = 0
        t0 = time.time()

        for i, (imgs, masks) in enumerate(train_loader):
            if i == 0:
                print("First batch loaded - GPU active")

            imgs, masks = imgs.to(DEVICE), masks.to(DEVICE)
            optimizer.zero_grad()

            try:
                with torch.amp.autocast('cuda'):
                    mask_logits, edge_logits, ds3, ds2 = model(imgs,
return_aux=True)
                    loss = loss_fn(mask_logits, edge_logits, masks,
epoch=epoch,
                                ds_logits=(ds3, ds2),
ds_weight=0.1)

                if torch.isnan(loss) or torch.isinf(loss):
                    nan_count += 1
                    print(f"NaN/Inf loss - batch {nan_count}")
                    if nan_count >= 3:
                        print("Too many NaN losses - breaking this
epoch")
                        break

```



```

        continue

        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(),
1.0)

        scaler.step(optimizer)
        scaler.update()
        train_loss += loss.item()

        if epoch == 0 and i == 0:
            with torch.no_grad():
                prob_mean =
torch.sigmoid(mask_logits[0]).mean().item()
                print(f'First batch mean prob: {prob_mean:.4f}
(>0.0 expected)')

        except RuntimeError as e:
            print(f"Batch error ({e}), skipping")
            continue

    # Validation every 2 epochs
    if epoch % 2 == 0 or epoch == epochs-1:
        model.eval(); dice_list = []
        with torch.no_grad():
            for imgs, masks in val_loader:
                imgs, masks = imgs.to(DEVICE), masks.to(DEVICE)
                with torch.amp.autocast('cuda'):
                    mask_logits, _ = model(imgs, return_aux=False)
                    preds = (torch.sigmoid(mask_logits) > 0.3).float()
                    inter = (preds*masks).sum(dim=(1,2,3))
                    union = preds.sum(dim=(1,2,3)) +
masks.sum(dim=(1,2,3))
                    dice = ((2*inter+1e-5)/(union+1e-5)).mean().item()
                    dice_list.append(dice)
            avg_dice = np.mean(dice_list)
            last_val_dice = avg_dice
        else:
            avg_dice = last_val_dice

    if epoch % 2 == 0 or epoch == epochs-1:
        if epoch >= warmup_epochs:
            scheduler.step(avg_dice)
        elif epoch == warmup_epochs-1:
            scheduler.step(avg_dice)

    duration = time.time()-t0
    print(f"Epoch {epoch+1:3d}/{epochs} | Loss
{train_loss/len(train_loader):.4f} | Val Dice {avg_dice:.4f} | LR
{optimizer.param_groups[0]['lr']:.2e} | {duration:.1f}s")

```

```

# Save checkpoint every epoch
torch.save({
    'epoch': epoch,
    'model_state': model.state_dict(),
    'optimizer_state': optimizer.state_dict(),
    'scheduler_state': scheduler.state_dict(),
    'scaler_state': scaler.state_dict(),
    'best_dice': best_dice,
    'patience_counter': counter
}, CKPT_PATH)

if epoch % 2 == 0 or epoch == epochs-1:
    if avg_dice > best_dice:
        best_dice = avg_dice; counter = 0
        torch.save(model.state_dict(), BEST_PATH)
        print(f' New best Dice {best_dice:.4f}')
    else:
        counter += 1
        if counter >= patience:
            print(f'Early stopping after {epoch+1} epochs.')
            break

    model.load_state_dict(torch.load(BEST_PATH, map_location=DEVICE,
weights_only=False))
    return model

# ===== MAIN =====
if __name__ == "__main__":
    BASE = '/kaggle/input/datasets/lakshaymiddha/crack-segmentation-
dataset/crack_segmentation_dataset'
    TRAIN_IMG = os.path.join(BASE, 'train', 'images')
    TRAIN_MASK= os.path.join(BASE, 'train', 'masks')
    TEST_IMG  = os.path.join(BASE, 'test', 'images')
    TEST_MASK = os.path.join(BASE, 'test', 'masks')

    print(f"Train images: {len(glob(os.path.join(TRAIN_IMG, '*')))}")
    print(f"Test images:  {len(glob(os.path.join(TEST_IMG, '*')))}")

    IMG_SIZE, BATCH, EPOCHS, PATIENCE = 320, 8, 100, 50

    train_ds = CrackDataset(TRAIN_IMG, TRAIN_MASK, size=(IMG_SIZE,
IMG_SIZE), augment=True)
    test_ds  = CrackDataset(TEST_IMG,  TEST_MASK,  size=(IMG_SIZE,
IMG_SIZE), augment=False)

    train_loader = DataLoader(train_ds, batch_size=BATCH,
shuffle=True, num_workers=0, pin_memory=False)
    test_loader  = DataLoader(test_ds, batch_size=BATCH,
shuffle=False, num_workers=0, pin_memory=False)

```

```

    model = CrackProMax(input_size=IMG_SIZE).to(DEVICE)
    print(f'Parameters: {sum(p.numel() for p in
model.parameters()):,}')

    model = train_model(train_loader, test_loader, model,
epochs=EPOCHS, patience=PATIENCE)
    print("Training finished - best model saved.")

```

No checkpoint - starting from scratch

Train images: 9603
Test images: 1695
Parameters: 3,685,484
Warm-up epoch 1: LR=1.00e-04
First batch loaded - GPU active
First batch mean prob: 0.4662 (>0.0 expected)
Epoch 1/100 | Loss 0.3975 | Val Dice 0.4311 | LR 1.00e-04 | 455.5s
New best Dice 0.4311
Warm-up epoch 2: LR=2.00e-04
First batch loaded - GPU active
Epoch 2/100 | Loss 0.3348 | Val Dice 0.4311 | LR 2.00e-04 | 161.6s
Warm-up epoch 3: LR=3.00e-04
First batch loaded - GPU active
Epoch 3/100 | Loss 0.3078 | Val Dice 0.5974 | LR 3.00e-04 | 177.9s
New best Dice 0.5974
Warm-up epoch 4: LR=4.00e-04
First batch loaded - GPU active
Epoch 4/100 | Loss 0.2967 | Val Dice 0.5974 | LR 4.00e-04 | 160.0s
Warm-up epoch 5: LR=5.00e-04
First batch loaded - GPU active
Epoch 6/100 | Loss 0.2749 | Val Dice 0.5438 | LR 5.00e-04 | 161.5s
First batch loaded - GPU active
Epoch 7/100 | Loss 0.2660 | Val Dice 0.4789 | LR 5.00e-04 | 176.7s
First batch loaded - GPU active
Epoch 8/100 | Loss 0.2592 | Val Dice 0.4789 | LR 5.00e-04 | 158.9s
First batch loaded - GPU active
Epoch 9/100 | Loss 0.2541 | Val Dice 0.6493 | LR 5.00e-04 | 176.6s
New best Dice 0.6493
First batch loaded - GPU active
Epoch 10/100 | Loss 0.2470 | Val Dice 0.6493 | LR 5.00e-04 | 160.4s
First batch loaded - GPU active
Epoch 11/100 | Loss 0.2453 | Val Dice 0.6369 | LR 5.00e-04 | 173.8s
First batch loaded - GPU active
Epoch 12/100 | Loss 0.2406 | Val Dice 0.6369 | LR 5.00e-04 | 158.5s
First batch loaded - GPU active
Epoch 13/100 | Loss 0.2368 | Val Dice 0.6272 | LR 5.00e-04 | 174.1s
First batch loaded - GPU active
Epoch 14/100 | Loss 0.2351 | Val Dice 0.6272 | LR 5.00e-04 | 158.4s
First batch loaded - GPU active
Epoch 15/100 | Loss 0.2311 | Val Dice 0.6264 | LR 5.00e-04 | 184.2s

```
First batch loaded - GPU active
Epoch 16/100 | Loss 0.2275 | Val Dice 0.6264 | LR 5.00e-04 | 163.0s
First batch loaded - GPU active
Epoch 17/100 | Loss 0.2260 | Val Dice 0.6345 | LR 5.00e-04 | 181.4s
First batch loaded - GPU active
Epoch 18/100 | Loss 0.2238 | Val Dice 0.6345 | LR 5.00e-04 | 158.5s
First batch loaded - GPU active
Epoch 19/100 | Loss 0.2202 | Val Dice 0.6687 | LR 5.00e-04 | 174.6s
  New best Dice 0.6687
First batch loaded - GPU active
Epoch 20/100 | Loss 0.2187 | Val Dice 0.6687 | LR 5.00e-04 | 158.0s
First batch loaded - GPU active
Epoch 21/100 | Loss 0.2156 | Val Dice 0.6767 | LR 5.00e-04 | 185.3s
  New best Dice 0.6767
First batch loaded - GPU active
Epoch 22/100 | Loss 0.2137 | Val Dice 0.6767 | LR 5.00e-04 | 157.9s
First batch loaded - GPU active
Epoch 23/100 | Loss 0.2117 | Val Dice 0.6749 | LR 5.00e-04 | 173.3s
First batch loaded - GPU active
Epoch 24/100 | Loss 0.2106 | Val Dice 0.6749 | LR 5.00e-04 | 156.9s
First batch loaded - GPU active
Epoch 25/100 | Loss 0.2085 | Val Dice 0.6646 | LR 5.00e-04 | 183.9s
First batch loaded - GPU active
Epoch 26/100 | Loss 0.2055 | Val Dice 0.6646 | LR 5.00e-04 | 161.6s
First batch loaded - GPU active
Epoch 27/100 | Loss 0.2057 | Val Dice 0.6024 | LR 5.00e-04 | 176.8s
First batch loaded - GPU active
Epoch 28/100 | Loss 0.2047 | Val Dice 0.6024 | LR 5.00e-04 | 160.6s
First batch loaded - GPU active
Epoch 29/100 | Loss 0.2016 | Val Dice 0.6569 | LR 5.00e-04 | 177.8s
First batch loaded - GPU active
Epoch 30/100 | Loss 0.1999 | Val Dice 0.6569 | LR 5.00e-04 | 157.0s
First batch loaded - GPU active
Epoch 31/100 | Loss 0.1995 | Val Dice 0.6829 | LR 5.00e-04 | 174.8s
  New best Dice 0.6829
First batch loaded - GPU active
Epoch 32/100 | Loss 0.1977 | Val Dice 0.6829 | LR 5.00e-04 | 158.8s
First batch loaded - GPU active
Epoch 33/100 | Loss 0.1982 | Val Dice 0.6782 | LR 5.00e-04 | 182.2s
First batch loaded - GPU active
Epoch 34/100 | Loss 0.1949 | Val Dice 0.6782 | LR 5.00e-04 | 158.2s
First batch loaded - GPU active
Epoch 35/100 | Loss 0.1949 | Val Dice 0.6765 | LR 5.00e-04 | 175.5s
First batch loaded - GPU active
Epoch 36/100 | Loss 0.1938 | Val Dice 0.6765 | LR 5.00e-04 | 196.3s
First batch loaded - GPU active
Epoch 37/100 | Loss 0.1920 | Val Dice 0.6818 | LR 5.00e-04 | 179.7s
First batch loaded - GPU active
Epoch 38/100 | Loss 0.1920 | Val Dice 0.6818 | LR 5.00e-04 | 162.9s
```

```

First batch loaded - GPU active
Epoch 39/100 | Loss 0.1909 | Val Dice 0.6903 | LR 5.00e-04 | 174.3s
New best Dice 0.6903
First batch loaded - GPU active
Epoch 40/100 | Loss 0.1890 | Val Dice 0.6903 | LR 5.00e-04 | 158.1s
First batch loaded - GPU active
Epoch 41/100 | Loss 0.1884 | Val Dice 0.6841 | LR 5.00e-04 | 174.9s
First batch loaded - GPU active
Epoch 42/100 | Loss 0.1866 | Val Dice 0.6841 | LR 5.00e-04 | 159.5s
First batch loaded - GPU active
Epoch 43/100 | Loss 0.1863 | Val Dice 0.6714 | LR 5.00e-04 | 175.8s
First batch loaded - GPU active
Epoch 44/100 | Loss 0.1855 | Val Dice 0.6714 | LR 5.00e-04 | 159.8s
First batch loaded - GPU active
Epoch 45/100 | Loss 0.1846 | Val Dice 0.6826 | LR 5.00e-04 | 204.7s
First batch loaded - GPU active
Epoch 46/100 | Loss 0.1831 | Val Dice 0.6826 | LR 5.00e-04 | 168.5s
First batch loaded - GPU active
Epoch 47/100 | Loss 0.1838 | Val Dice 0.6838 | LR 5.00e-04 | 194.0s
First batch loaded - GPU active
Epoch 48/100 | Loss 0.1814 | Val Dice 0.6838 | LR 5.00e-04 | 164.6s
First batch loaded - GPU active
Epoch 49/100 | Loss 0.1801 | Val Dice 0.6861 | LR 5.00e-04 | 178.1s
First batch loaded - GPU active
Epoch 50/100 | Loss 0.1800 | Val Dice 0.6861 | LR 5.00e-04 | 161.2s
First batch loaded - GPU active
Epoch 51/100 | Loss 0.1805 | Val Dice 0.6895 | LR 2.50e-04 | 176.1s
First batch loaded - GPU active
Epoch 52/100 | Loss 0.1724 | Val Dice 0.6895 | LR 2.50e-04 | 157.3s
First batch loaded - GPU active
Epoch 53/100 | Loss 0.1716 | Val Dice 0.6941 | LR 2.50e-04 | 178.6s
New best Dice 0.6941
First batch loaded - GPU active
Epoch 54/100 | Loss 0.1705 | Val Dice 0.6941 | LR 2.50e-04 | 159.2s
First batch loaded - GPU active
Epoch 55/100 | Loss 0.1688 | Val Dice 0.6958 | LR 2.50e-04 | 175.5s
New best Dice 0.6958
First batch loaded - GPU active
Epoch 56/100 | Loss 0.1681 | Val Dice 0.6958 | LR 2.50e-04 | 157.6s
First batch loaded - GPU active
Epoch 57/100 | Loss 0.1669 | Val Dice 0.6977 | LR 2.50e-04 | 174.2s
New best Dice 0.6977
First batch loaded - GPU active
Epoch 58/100 | Loss 0.1663 | Val Dice 0.6977 | LR 2.50e-04 | 160.5s
First batch loaded - GPU active
Epoch 59/100 | Loss 0.1654 | Val Dice 0.6993 | LR 2.50e-04 | 177.1s
New best Dice 0.6993
First batch loaded - GPU active
Epoch 60/100 | Loss 0.1644 | Val Dice 0.6993 | LR 2.50e-04 | 157.8s

```

```
First batch loaded - GPU active
Epoch 61/100 | Loss 0.1636 | Val Dice 0.7009 | LR 2.50e-04 | 174.2s
New best Dice 0.7009
First batch loaded - GPU active
Epoch 62/100 | Loss 0.1630 | Val Dice 0.7009 | LR 2.50e-04 | 164.4s
First batch loaded - GPU active
Epoch 63/100 | Loss 0.1617 | Val Dice 0.6979 | LR 2.50e-04 | 181.4s
First batch loaded - GPU active
Epoch 64/100 | Loss 0.1620 | Val Dice 0.6979 | LR 2.50e-04 | 163.7s
First batch loaded - GPU active
Epoch 65/100 | Loss 0.1613 | Val Dice 0.6977 | LR 2.50e-04 | 184.5s
First batch loaded - GPU active
Epoch 66/100 | Loss 0.1612 | Val Dice 0.6977 | LR 2.50e-04 | 159.3s
First batch loaded - GPU active
Epoch 67/100 | Loss 0.1601 | Val Dice 0.7033 | LR 2.50e-04 | 178.9s
New best Dice 0.7033
First batch loaded - GPU active
Epoch 68/100 | Loss 0.1600 | Val Dice 0.7033 | LR 2.50e-04 | 159.2s
First batch loaded - GPU active
Epoch 69/100 | Loss 0.1594 | Val Dice 0.6990 | LR 2.50e-04 | 173.9s
First batch loaded - GPU active
Epoch 70/100 | Loss 0.1587 | Val Dice 0.6990 | LR 2.50e-04 | 157.1s
First batch loaded - GPU active
Epoch 71/100 | Loss 0.1579 | Val Dice 0.6950 | LR 2.50e-04 | 174.4s
First batch loaded - GPU active
Epoch 72/100 | Loss 0.1573 | Val Dice 0.6950 | LR 2.50e-04 | 160.1s
First batch loaded - GPU active
Epoch 73/100 | Loss 0.1566 | Val Dice 0.6960 | LR 2.50e-04 | 176.5s
First batch loaded - GPU active
Epoch 74/100 | Loss 0.1563 | Val Dice 0.6960 | LR 2.50e-04 | 159.3s
First batch loaded - GPU active
Epoch 75/100 | Loss 0.1562 | Val Dice 0.6974 | LR 2.50e-04 | 179.2s
First batch loaded - GPU active
Epoch 76/100 | Loss 0.1550 | Val Dice 0.6974 | LR 2.50e-04 | 163.1s
First batch loaded - GPU active
Epoch 77/100 | Loss 0.1553 | Val Dice 0.6975 | LR 2.50e-04 | 178.3s
First batch loaded - GPU active
Epoch 78/100 | Loss 0.1545 | Val Dice 0.6975 | LR 2.50e-04 | 161.3s
First batch loaded - GPU active
Epoch 79/100 | Loss 0.1541 | Val Dice 0.6969 | LR 1.25e-04 | 175.2s
First batch loaded - GPU active
Epoch 80/100 | Loss 0.1511 | Val Dice 0.6969 | LR 1.25e-04 | 157.0s
First batch loaded - GPU active
Epoch 81/100 | Loss 0.1502 | Val Dice 0.6976 | LR 1.25e-04 | 175.5s
First batch loaded - GPU active
Epoch 82/100 | Loss 0.1491 | Val Dice 0.6976 | LR 1.25e-04 | 157.6s
First batch loaded - GPU active
Epoch 83/100 | Loss 0.1486 | Val Dice 0.7067 | LR 1.25e-04 | 174.9s
New best Dice 0.7067
```

```

First batch loaded - GPU active
Epoch 84/100 | Loss 0.1475 | Val Dice 0.7067 | LR 1.25e-04 | 159.3s
First batch loaded - GPU active
Epoch 85/100 | Loss 0.1469 | Val Dice 0.7058 | LR 1.25e-04 | 176.3s
First batch loaded - GPU active
Epoch 86/100 | Loss 0.1467 | Val Dice 0.7058 | LR 1.25e-04 | 158.2s
First batch loaded - GPU active
Epoch 87/100 | Loss 0.1463 | Val Dice 0.7026 | LR 1.25e-04 | 175.2s
First batch loaded - GPU active
Epoch 88/100 | Loss 0.1457 | Val Dice 0.7026 | LR 1.25e-04 | 159.9s
First batch loaded - GPU active
Epoch 89/100 | Loss 0.1454 | Val Dice 0.7013 | LR 1.25e-04 | 174.4s
First batch loaded - GPU active
Epoch 90/100 | Loss 0.1449 | Val Dice 0.7013 | LR 1.25e-04 | 158.8s
First batch loaded - GPU active
Epoch 91/100 | Loss 0.1443 | Val Dice 0.6986 | LR 1.25e-04 | 177.5s
First batch loaded - GPU active
Epoch 92/100 | Loss 0.1447 | Val Dice 0.6986 | LR 1.25e-04 | 217.5s
First batch loaded - GPU active
Epoch 93/100 | Loss 0.1443 | Val Dice 0.7044 | LR 1.25e-04 | 327.5s
First batch loaded - GPU active
Epoch 94/100 | Loss 0.1434 | Val Dice 0.7044 | LR 1.25e-04 | 162.7s
First batch loaded - GPU active
Epoch 95/100 | Loss 0.1431 | Val Dice 0.7008 | LR 6.25e-05 | 180.3s
First batch loaded - GPU active
Epoch 96/100 | Loss 0.1416 | Val Dice 0.7008 | LR 6.25e-05 | 165.1s
First batch loaded - GPU active
Epoch 97/100 | Loss 0.1409 | Val Dice 0.7056 | LR 6.25e-05 | 176.2s
First batch loaded - GPU active
Epoch 98/100 | Loss 0.1404 | Val Dice 0.7056 | LR 6.25e-05 | 157.1s
First batch loaded - GPU active
Epoch 99/100 | Loss 0.1399 | Val Dice 0.7035 | LR 6.25e-05 | 175.7s
First batch loaded - GPU active
Epoch 100/100 | Loss 0.1395 | Val Dice 0.7048 | LR 6.25e-05 | 176.5s
Training finished - best model saved.

```

```

import os, cv2, numpy as np, torch, torch.nn as nn,
torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from glob import glob
import random, time

# We reuse the same CrackProMax architecture, but with input_size=512
# (The class is already defined, but we redefine it here with the new
# default just to be safe)

# ----- SAME MODEL CLASS, BUT input_size DEFAULT 512 -----
class CrackProMax(nn.Module):
    def __init__(self, input_size=512):
        super().__init__()

```

```

self.input_size = input_size

# (Copy the entire __init__ from your original training cell
here)
# Make sure it is identical to the definition you used for
320,
# only change the default value of input_size to 512.
# I'll provide the full class below for completeness.
self.stem = nn.Sequential(
    nn.Conv2d(3, 32, 7, stride=2, padding=3, bias=False),
    norm_layer(32), nn.ReLU(inplace=True))

self.enc1 = nn.Sequential(
    AOSC(32),
    nn.Conv2d(32, 64, 3, padding=1), norm_layer(64),
nn.ReLU(inplace=True),
    nn.MaxPool2d(2))

self.enc2 = nn.Sequential(
    nn.Conv2d(64, 128, 3, padding=1), norm_layer(128),
nn.ReLU(inplace=True),
    nn.MaxPool2d(2))

self.enc3 = nn.Sequential(
    nn.Conv2d(128, 256, 3, padding=1), norm_layer(256),
nn.ReLU(inplace=True),
    nn.MaxPool2d(2))

self.enc4 = nn.Sequential(
    nn.Conv2d(256, 512, 3, padding=1), norm_layer(512),
nn.ReLU(inplace=True),
    nn.MaxPool2d(2))

self.bottleneck = nn.Sequential(
    nn.Conv2d(512, 512, 3, padding=2, dilation=2, groups=512,
bias=False),
    nn.Conv2d(512, 512, 1), norm_layer(512),
nn.ReLU(inplace=True),
    SOTS(512))

self.up4 = nn.ConvTranspose2d(512, 256, 2, stride=2)
self.rcp4 = GatedRCP(256)
self.skip4 = nn.Conv2d(256+256, 256, 1)

self.up3 = nn.ConvTranspose2d(256, 128, 2, stride=2)
self.rcp3 = GatedRCP(128)
self.skip3 = nn.Conv2d(128+128, 128, 1)

self.up2 = nn.ConvTranspose2d(128, 64, 2, stride=2)
self.rcp2 = GatedRCP(64)

```



```

self.skip2 = nn.Conv2d(64+64, 64, 1)

self.up1 = nn.ConvTranspose2d(64, 32, 2, stride=2)
self.rcp1 = GatedRCP(32)
self.skip1 = nn.Conv2d(32+32, 32, 1)

self.ms_fusion = MultiScaleFusion([256, 128, 64, 32], 32)

self.mask_head = nn.Conv2d(32, 1, 1)
self.edge_head = nn.Conv2d(32, 1, 1)

self.ds_head3 = nn.Conv2d(128, 1, 1)
self.ds_head2 = nn.Conv2d(64, 1, 1)

def forward(self, x, return_aux=False):
    s0 = self.stem(x)
    e1 = self.enc1(s0)
    e2 = self.enc2(e1)
    e3 = self.enc3(e2)
    e4 = self.enc4(e3)
    b = self.bottleneck(e4)

    d4 = self.up4(b)
    d4 = self.rcp4(d4)
    d4 = self.skip4(torch.cat([d4, e3], 1))

    d3 = self.up3(d4)
    d3 = self.rcp3(d3)
    d3 = self.skip3(torch.cat([d3, e2], 1))

    d2 = self.up2(d3)
    d2 = self.rcp2(d2)
    d2 = self.skip2(torch.cat([d2, e1], 1))

    d1 = self.up1(d2)
    d1 = self.rcp1(d1)
    d1 = self.skip1(torch.cat([d1, s0], 1))

    fused = self.ms_fusion([d4, d3, d2, d1])

    mask_logits = self.mask_head(fused)
    edge_logits = self.edge_head(fused)

    if mask_logits.shape[2] != self.input_size:
        mask_logits = F.interpolate(mask_logits,
size=(self.input_size, self.input_size),
mode='bilinear',
align_corners=False)
        edge_logits = F.interpolate(edge_logits,
size=(self.input_size, self.input_size),

```

```

mode='bilinear',
align_corners=False)

    if return_aux:
        ds3 = self.ds_head3(d3)
        ds2 = self.ds_head2(d2)
        ds3 = F.interpolate(ds3, size=(self.input_size,
self.input_size), mode='bilinear', align_corners=False)
        ds2 = F.interpolate(ds2, size=(self.input_size,
self.input_size), mode='bilinear', align_corners=False)
        return mask_logits, edge_logits, ds3, ds2
    else:
        return mask_logits, edge_logits

# ----- LOSS FUNCTION (same as before, no cLDice) -----
def create_edge_target(mask):
    with torch.no_grad():
        sobel_x = torch.tensor([[[-1,0,1],[-2,0,2],[-1,0,1]],
device=mask.device, dtype=torch.float32).view(1,1,3,3)
        sobel_y = torch.tensor([[[-1,-2,-1],[0,0,0],[1,2,1]],
device=mask.device, dtype=torch.float32).view(1,1,3,3)
        edge = (F.conv2d(mask, sobel_x, padding=1).abs() +
F.conv2d(mask, sobel_y, padding=1).abs()).clamp(0,1)
    return edge

def focal_loss(logits, target, alpha=0.25, gamma=2.0,
pos_weight=10.0):
    prob = torch.sigmoid(logits).clamp(min=1e-7, max=1-1e-7)
    pt = prob*target + (1-prob)*(1-target)
    weight = alpha*target + (1-alpha)*(1-target)
    weight_pos = pos_weight*target + (1-target)
    loss = -weight_pos * weight * ((1-pt).pow(gamma)) * torch.log(pt)
    return loss.mean()

def dice_loss(prob, target):
    smooth = 1e-5
    num = 2*(prob*target).sum() + smooth
    den = prob.sum() + target.sum() + smooth
    return 1 - num/den

def loss_fn(mask_logits, edge_logits, mask_gt, epoch=0,
ds_logits=None, ds_weight=0.1):
    prob = torch.sigmoid(mask_logits).clamp(min=1e-7, max=1-1e-7)
    mask_loss = 0.7*focal_loss(mask_logits, mask_gt, pos_weight=15.0)
+ \
        0.3*dice_loss(prob, mask_gt)
    mask_loss = mask_loss * (1 + 2 * mask_gt.mean().clamp(min=0))
    edge_gt = create_edge_target(mask_gt)
    edge_loss = F.binary_cross_entropy_with_logits(edge_logits,
edge_gt)

```

```

total = mask_loss + 0.35*edge_loss
if ds_logits is not None:
    ds3_logits, ds2_logits = ds_logits
    if ds3_logits is not None:
        ds3_prob = torch.sigmoid(ds3_logits).clamp(min=1e-7,
max=1-1e-7)
        total += ds_weight * (0.5*focal_loss(ds3_logits, mask_gt,
pos_weight=10) +
                                0.5*dice_loss(ds3_prob, mask_gt))
    if ds2_logits is not None:
        ds2_prob = torch.sigmoid(ds2_logits).clamp(min=1e-7,
max=1-1e-7)
        total += ds_weight * (0.5*focal_loss(ds2_logits, mask_gt,
pos_weight=10) +
                                0.5*dice_loss(ds2_prob, mask_gt))
    return total

# ----- FINE-TUNING LOOP -----
def fine_tune(model, train_loader, val_loader, epochs=30):
    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4,
weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer,
mode='max', factor=0.5, patience=3)
    scaler = torch.amp.GradScaler('cuda')
    best_dice = 0.0

    for epoch in range(epochs):
        model.train(); train_loss = 0.0
        for imgs, masks in train_loader:
            imgs, masks = imgs.to(DEVICE), masks.to(DEVICE)
            optimizer.zero_grad()
            with torch.amp.autocast('cuda'):
                mask_logits, edge_logits, ds3, ds2 = model(imgs,
return_aux=True)
                loss = loss_fn(mask_logits, edge_logits, masks,
epoch=epoch,
                                ds_logits=(ds3, ds2), ds_weight=0.1)
            scaler.scale(loss).backward()
            scaler.unscale_(optimizer)
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            scaler.step(optimizer)
            scaler.update()
            train_loss += loss.item()

        # Validation
        model.eval(); dice_list = []
        with torch.no_grad():
            for imgs, masks in val_loader:
                imgs, masks = imgs.to(DEVICE), masks.to(DEVICE)
                with torch.amp.autocast('cuda'):

```

```

        mask_logits, _ = model(imgs, return_aux=False)
        preds = (torch.sigmoid(mask_logits) > 0.3).float()
        inter = (preds*masks).sum(dim=(1,2,3))
        union = preds.sum(dim=(1,2,3)) +
masks.sum(dim=(1,2,3))
        dice = ((2*inter+1e-5)/(union+1e-5)).mean().item()
        dice_list.append(dice)
        avg_dice = np.mean(dice_list)
        scheduler.step(avg_dice)

        if avg_dice > best_dice:
            best_dice = avg_dice
            torch.save(model.state_dict(),
'/kaggle/working/best_model_512.pth')
            print(f"    New best Dice {best_dice:.4f}")

            print(f"Fine-tune epoch {epoch+1}/{epochs} | Loss
{train_loss/len(train_loader):.4f} | Val Dice {avg_dice:.4f}")

        return best_dice

# ----- MAIN -----
if __name__ == "__main__":
    # Load the best 320 model
    best_320_path = '/kaggle/working/best_model.pth'
    model_512 = CrackProMax(input_size=512).to(DEVICE)
    model_512.load_state_dict(torch.load(best_320_path,
map_location=DEVICE, weights_only=False))
    print("\n Loaded best 320 model for fine-tuning")

    BASE = '/kaggle/input/datasets/lakshaymiddha/crack-segmentation-
dataset/crack_segmentation_dataset'
    TRAIN_IMG = os.path.join(BASE, 'train', 'images')
    TRAIN_MASK= os.path.join(BASE, 'train', 'masks')
    TEST_IMG  = os.path.join(BASE, 'test', 'images')
    TEST_MASK = os.path.join(BASE, 'test', 'masks')

    # Dataloaders for 512x512 (small batch)
    train_ds = CrackDataset(TRAIN_IMG, TRAIN_MASK, size=(512,512),
augment=True)
    test_ds  = CrackDataset(TEST_IMG,  TEST_MASK,  size=(512,512),
augment=False)

    train_loader = DataLoader(train_ds, batch_size=2, shuffle=True,
num_workers=0, pin_memory=True)
    test_loader  = DataLoader(test_ds,  batch_size=2, shuffle=False,
num_workers=0, pin_memory=True)

    print("Starting fine-tuning on 512x512...")
    final_dice = fine_tune(model_512, train_loader, test_loader,

```

```
epochs=30)
    print(f" Fine-tuning complete. Best validation Dice:
{final_dice:.4f}")
```

```
 Loaded best 320 model for fine-tuning
```

```
Starting fine-tuning on 512x512...
```

```
    New best Dice 0.6959
```

```
Fine-tune epoch 1/30 | Loss 0.2144 | Val Dice 0.6959
```

```
    New best Dice 0.7014
```

```
Fine-tune epoch 2/30 | Loss 0.2031 | Val Dice 0.7014
```

```
    New best Dice 0.7113
```

```
Fine-tune epoch 3/30 | Loss 0.2004 | Val Dice 0.7113
```

```
Fine-tune epoch 4/30 | Loss 0.1968 | Val Dice 0.7067
```

```
Fine-tune epoch 5/30 | Loss 0.1950 | Val Dice 0.7058
```

```
Fine-tune epoch 6/30 | Loss 0.1930 | Val Dice 0.7113
```

```
Fine-tune epoch 7/30 | Loss 0.1904 | Val Dice 0.7093
```

```
Fine-tune epoch 8/30 | Loss 0.1877 | Val Dice 0.7057
```

```
Fine-tune epoch 9/30 | Loss 0.1852 | Val Dice 0.7112
```

```
Fine-tune epoch 10/30 | Loss 0.1843 | Val Dice 0.7083
```

```
Fine-tune epoch 11/30 | Loss 0.1827 | Val Dice 0.7086
```

```
    New best Dice 0.7116
```

```
Fine-tune epoch 12/30 | Loss 0.1806 | Val Dice 0.7116
```

```
    New best Dice 0.7137
```

```
Fine-tune epoch 13/30 | Loss 0.1802 | Val Dice 0.7137
```

```
Fine-tune epoch 14/30 | Loss 0.1785 | Val Dice 0.7074
```

```
    New best Dice 0.7142
```

```
Fine-tune epoch 15/30 | Loss 0.1785 | Val Dice 0.7142
```

```
Fine-tune epoch 16/30 | Loss 0.1790 | Val Dice 0.7126
```

```
Fine-tune epoch 17/30 | Loss 0.1775 | Val Dice 0.7084
```

```
Fine-tune epoch 18/30 | Loss 0.1777 | Val Dice 0.7105
```

```
Fine-tune epoch 19/30 | Loss 0.1769 | Val Dice 0.7123
```

```
Fine-tune epoch 20/30 | Loss 0.1762 | Val Dice 0.7128
```

```
    New best Dice 0.7151
```

```
Fine-tune epoch 21/30 | Loss 0.1767 | Val Dice 0.7151
```

```
Fine-tune epoch 22/30 | Loss 0.1755 | Val Dice 0.7117
```

```
Fine-tune epoch 23/30 | Loss 0.1746 | Val Dice 0.7137
```

```
    New best Dice 0.7163
```

```
Fine-tune epoch 24/30 | Loss 0.1753 | Val Dice 0.7163
```

```
Fine-tune epoch 25/30 | Loss 0.1748 | Val Dice 0.7131
```

```
Fine-tune epoch 26/30 | Loss 0.1752 | Val Dice 0.7109
```

```
Fine-tune epoch 27/30 | Loss 0.1739 | Val Dice 0.7124
```

```
Fine-tune epoch 28/30 | Loss 0.1736 | Val Dice 0.7153
```

```
Fine-tune epoch 29/30 | Loss 0.1742 | Val Dice 0.7111
```

```
Fine-tune epoch 30/30 | Loss 0.1726 | Val Dice 0.7128
```

```
 Fine-tuning complete. Best validation Dice: 0.7163
```

```
import torch, numpy as np, matplotlib.pyplot as plt, cv2, random, os
from torch.utils.data import DataLoader
```

```
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```

best_model_path = '/kaggle/working/best_model_512.pth'

# Load the 512 model
model = CrackProMax(input_size=512).to(DEVICE)
model.load_state_dict(torch.load(best_model_path, map_location=DEVICE,
weights_only=False))
model.eval()
print("✅ Best 512 model loaded")

# Data paths
BASE = '/kaggle/input/datasets/lakshaymiddha/crack-segmentation-
dataset/crack_segmentation_dataset'
TEST_IMG = os.path.join(BASE, 'test', 'images')
TEST_MASK = os.path.join(BASE, 'test', 'masks')
test_ds = CrackDataset(TEST_IMG, TEST_MASK, size=(512,512),
augment=False)
test_loader = DataLoader(test_ds, batch_size=1, shuffle=False)

# ----- FIXED STRONG TTA (6 views) -----
@torch.no_grad()
def tta_predict(img_tensor, threshold=0.3):
    """img_tensor: [1, C, H, W]"""
    img = img_tensor.to(DEVICE)
    preds = []

    # 1. Original
    preds.append(torch.sigmoid(model(img, return_aux=False)[0]))

    # 2. Horizontal flip
    preds.append(torch.sigmoid(model(img.flip(-1), return_aux=False)
[0]).flip(-1))

    # 3. Vertical flip
    preds.append(torch.sigmoid(model(img.flip(-2), return_aux=False)
[0]).flip(-2))

    # 4-6. Rotations (90, 180, 270) – correctly rotate and rotate back
    for k in [1, 2, -1]:
        rot_img = torch.rot90(img, k=k, dims=[2,3])          # rotate
image spatial dims
        p = torch.sigmoid(model(rot_img, return_aux=False)[0])
        p_back = torch.rot90(p, k=-k, dims=[2,3])           # rotate
back to original orientation
        preds.append(p_back)

    prob = torch.stack(preds, dim=0).mean(dim=0)    # average over 6
views
    return (prob > threshold).float().cpu().squeeze() # [H, W]

# ----- POST-PROCESSING -----

```

```

def post_process(mask_tensor, min_size=20, close_kernel=5):
    """mask_tensor: [H, W] on CPU"""
    mask = (mask_tensor.numpy()*255).astype(np.uint8)
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE,
    (close_kernel, close_kernel))
    closed = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
    num, labels, stats, _ = cv2.connectedComponentsWithStats(closed,
    connectivity=8)
    out = np.zeros_like(closed)
    for j in range(1, num):
        if stats[j, cv2.CC_STAT_AREA] >= min_size:
            out[labels == j] = 255
    return torch.from_numpy(out/255.0).unsqueeze(0) # [1, H, W]

# ----- METRICS -----
def compute_metrics(pred, mask):
    p = pred.view(-1); m = mask.view(-1)
    tp = (p*m).sum(); fp = (p*(1-m)).sum(); fn = ((1-p)*m).sum()
    iou = tp/(tp+fp+fn+1e-7)
    dice = (2*tp)/(2*tp+fp+fn+1e-7)
    prec = tp/(tp+fp+1e-7); rec = tp/(tp+fn+1e-7)
    f1 = 2*prec*rec/(prec+rec+1e-7)
    return iou.item(), dice.item(), prec.item(), rec.item(), f1.item()

def evaluate(loader, threshold):
    iou_l, dice_l, prec_l, rec_l, f1_l = [], [], [], [], []
    for img, mask in loader:
        pred_raw = tta_predict(img, threshold)
        pred = post_process(pred_raw)
        iou, dice, prec, rec, f1 = compute_metrics(pred.cpu(),
        mask.cpu())
        iou_l.append(iou); dice_l.append(dice); prec_l.append(prec);
        rec_l.append(rec); f1_l.append(f1)
    return np.mean(iou_l), np.mean(dice_l), np.mean(prec_l),
    np.mean(rec_l), np.mean(f1_l)

# ----- THRESHOLD SWEEP -----
print("\n Threshold sweep (STRONG TTA + post-processing):")
best_dice, best_thr = 0, 0.3
for t in np.arange(0.2, 0.7, 0.05):
    iou, dice, prec, rec, f1 = evaluate(test_loader, t)
    print(f" {t:.2f} IoU {iou:.4f} Dice {dice:.4f} Prec {prec:.4f}
    Rec {rec:.4f} F1 {f1:.4f}")
    if dice > best_dice:
        best_dice, best_thr = dice, t
print(f"\n Best threshold: {best_thr:.2f} → Dice {best_dice:.4f}")

# ----- FINAL METRICS -----
iou, dice, prec, rec, f1 = evaluate(test_loader, best_thr)
print(f"\n FINAL RESULTS (Strong TTA + post-processing):")

```

```

print(f" IoU      : {iou:.4f}")
print(f" Dice     : {dice:.4f}")
print(f" Precision : {prec:.4f}")
print(f" Recall    : {rec:.4f}")
print(f" F1 Score   : {f1:.4f}")

# ----- VISUALISATION -----
def show_predictions(num=4):
    indices = random.sample(range(len(test_ds)), num)
    fig, axes = plt.subplots(num, 3, figsize=(12, num*4))
    for i, idx in enumerate(indices):
        img, mask = test_ds[idx]
        pred_raw = tta_predict(img.unsqueeze(0), best_thr)
        pred = post_process(pred_raw).squeeze().numpy()
        axes[i,0].imshow(img.permute(1,2,0));
    axes[i,0].set_title('Input'); axes[i,0].axis('off')
    axes[i,1].imshow(mask.squeeze(), cmap='gray');
    axes[i,1].set_title('Ground Truth'); axes[i,1].axis('off')
    axes[i,2].imshow(pred, cmap='gray');
    axes[i,2].set_title('Prediction'); axes[i,2].axis('off')
    plt.tight_layout(); plt.show()

show_predictions(4)

```

□ Best 512 model loaded

□ Threshold sweep (STRONG TTA + post-processing):

0.20	IoU	0.4386	Dice	0.5701	Prec	0.4677	Rec	0.7869	F1	0.5701
0.25	IoU	0.4523	Dice	0.5823	Prec	0.4897	Rec	0.7701	F1	0.5823
0.30	IoU	0.4635	Dice	0.5921	Prec	0.5103	Rec	0.7526	F1	0.5921
0.35	IoU	0.4722	Dice	0.5995	Prec	0.5300	Rec	0.7336	F1	0.5995
0.40	IoU	0.4785	Dice	0.6046	Prec	0.5474	Rec	0.7147	F1	0.6046
0.45	IoU	0.4828	Dice	0.6079	Prec	0.5647	Rec	0.6952	F1	0.6079
0.50	IoU	0.4849	Dice	0.6093	Prec	0.5810	Rec	0.6747	F1	0.6093
0.55	IoU	0.4848	Dice	0.6087	Prec	0.5975	Rec	0.6533	F1	0.6087
0.60	IoU	0.4821	Dice	0.6057	Prec	0.6139	Rec	0.6302	F1	0.6057
0.65	IoU	0.4744	Dice	0.5977	Prec	0.6290	Rec	0.6026	F1	0.5977

□ Best threshold: 0.50 → Dice 0.6093

□ FINAL RESULTS (Strong TTA + post-processing):

```

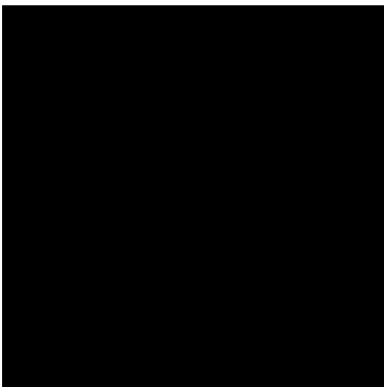
IoU      : 0.4849
Dice     : 0.6093
Precision : 0.5810
Recall    : 0.6747
F1 Score  : 0.6093

```

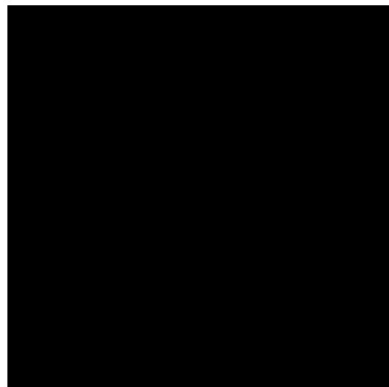

Input



Ground Truth



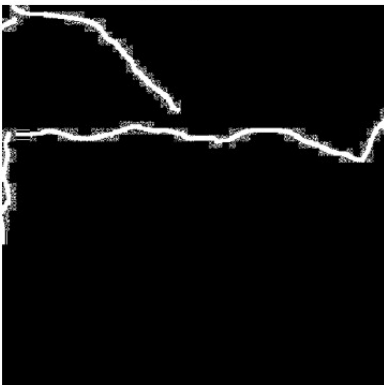
Prediction



Input



Ground Truth



Prediction



Input



Ground Truth



Prediction



Input



Ground Truth



Prediction



```

import torch, numpy as np
from torch.utils.data import DataLoader

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.eval()
test_loader = DataLoader(test_ds, batch_size=1, shuffle=False)

dices = []
with torch.no_grad():
    for img, mask in test_loader:
        img = img.to(DEVICE)
        mask = mask.cpu()
        logits, _ = model(img, return_aux=False)
        pred = (torch.sigmoid(logits) > 0.3).float().cpu()
        inter = (pred * mask).sum()
        union = pred.sum() + mask.sum()
        dice = (2*inter + 1e-7) / (union + 1e-7)
        dices.append(dice.item())

raw_dice = np.mean(dices)
print(f"Raw Dice (no TTA, no post-processing): {raw_dice:.4f}")

Raw Dice (no TTA, no post-processing): 0.7161

import torch, numpy as np, matplotlib.pyplot as plt, cv2, random, os
from torch.utils.data import DataLoader

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
best_model_path = '/kaggle/working/best_model_512.pth'

model = CrackProMax(input_size=512).to(DEVICE)
model.load_state_dict(torch.load(best_model_path, map_location=DEVICE,
weights_only=False))
model.eval()
print("✅ Best 512 model loaded")

BASE = '/kaggle/input/datasets/lakshaymiddha/crack-segmentation-
dataset/crack_segmentation_dataset'
TEST_IMG = os.path.join(BASE, 'test', 'images')
TEST_MASK = os.path.join(BASE, 'test', 'masks')
test_ds = CrackDataset(TEST_IMG, TEST_MASK, size=(512,512),
augment=False)
test_loader = DataLoader(test_ds, batch_size=1, shuffle=False)

# ----- FLIP-ONLY TTA (known to the model) -----
@torch.no_grad()
def tta_predict(img_tensor, threshold=0.3):
    img = img_tensor.to(DEVICE)
    # Original
    p0 = torch.sigmoid(model(img, return_aux=False)[0])

```

```

    # Horizontal flip
    p_h = torch.sigmoid(model(img.flip(-1), return_aux=False)
[0]).flip(-1)
    # Vertical flip
    p_v = torch.sigmoid(model(img.flip(-2), return_aux=False)
[0]).flip(-2)
    prob = (p0 + p_h + p_v) / 3
    return (prob > threshold).float().cpu().squeeze()    # [H, W]

# ----- POST-PROCESSING -----
def post_process(mask_tensor, min_size=20, close_kernel=5):
    mask = (mask_tensor.numpy()*255).astype(np.uint8)
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE,
(close_kernel, close_kernel))
    closed = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
    num, labels, stats, _ = cv2.connectedComponentsWithStats(closed,
connectivity=8)
    out = np.zeros_like(closed)
    for j in range(1, num):
        if stats[j, cv2.CC_STAT_AREA] >= min_size:
            out[labels == j] = 255
    return torch.from_numpy(out/255.0).unsqueeze(0)

# ----- METRICS -----
def compute_metrics(pred, mask):
    p = pred.view(-1); m = mask.view(-1)
    tp = (p*m).sum(); fp = (p*(1-m)).sum(); fn = ((1-p)*m).sum()
    iou = tp/(tp+fp+fn+1e-7)
    dice = (2*tp)/(2*tp+fp+fn+1e-7)
    prec = tp/(tp+fp+1e-7); rec = tp/(tp+fn+1e-7)
    f1 = 2*prec*rec/(prec+rec+1e-7)
    return iou.item(), dice.item(), prec.item(), rec.item(), f1.item()

def evaluate(loader, threshold):
    iou_l, dice_l, prec_l, rec_l, f1_l = [], [], [], [], []
    for img, mask in loader:
        pred_raw = tta_predict(img, threshold)
        pred = post_process(pred_raw)
        iou, dice, prec, rec, f1 = compute_metrics(pred.cpu(),
mask.cpu())
        iou_l.append(iou); dice_l.append(dice); prec_l.append(prec);
rec_l.append(rec); f1_l.append(f1)
    return np.mean(iou_l), np.mean(dice_l), np.mean(prec_l),
np.mean(rec_l), np.mean(f1_l)

# ----- THRESHOLD SWEEP -----
print("\n Threshold sweep (Flip TTA + post-processing):")
best_dice, best_thr = 0, 0.3
for t in np.arange(0.2, 0.7, 0.05):
    iou, dice, prec, rec, f1 = evaluate(test_loader, t)

```

```

    print(f" {t:.2f} IoU {iou:.4f} Dice {dice:.4f} Prec {prec:.4f}
Rec {rec:.4f} F1 {f1:.4f}")
    if dice > best_dice:
        best_dice, best_thr = dice, t
print(f"\n Best threshold: {best_thr:.2f} → Dice {best_dice:.4f}")

# ----- FINAL METRICS -----
iou, dice, prec, rec, f1 = evaluate(test_loader, best_thr)
print(f"\n FINAL RESULTS (Flip TTA + post-processing):")
print(f" IoU : {iou:.4f}")
print(f" Dice : {dice:.4f}")
print(f" Precision : {prec:.4f}")
print(f" Recall : {rec:.4f}")
print(f" F1 Score : {f1:.4f}")

# ----- VISUALISATION -----
def show_predictions(num=4):
    indices = random.sample(range(len(test_ds)), num)
    fig, axes = plt.subplots(num, 3, figsize=(12, num*4))
    for i, idx in enumerate(indices):
        img, mask = test_ds[idx]
        pred_raw = tta_predict(img.unsqueeze(0), best_thr)
        pred = post_process(pred_raw).squeeze().numpy()
        axes[i,0].imshow(img.permute(1,2,0))
    axes[i,0].set_title('Input'); axes[i,0].axis('off')
        axes[i,1].imshow(mask.squeeze(), cmap='gray');
    axes[i,1].set_title('Ground Truth'); axes[i,1].axis('off')
        axes[i,2].imshow(pred, cmap='gray');
    axes[i,2].set_title('Prediction'); axes[i,2].axis('off')
    plt.tight_layout(); plt.show()

show_predictions(4)

```

□ Best 512 model loaded

□ Threshold sweep (Flip TTA + post-processing):

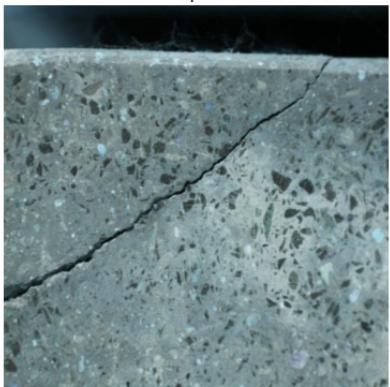
0.20	IoU	0.4417	Dice	0.5727	Prec	0.4723	Rec	0.7823	F1	0.5727
0.25	IoU	0.4534	Dice	0.5830	Prec	0.4910	Rec	0.7671	F1	0.5830
0.30	IoU	0.4631	Dice	0.5913	Prec	0.5090	Rec	0.7510	F1	0.5913
0.35	IoU	0.4709	Dice	0.5980	Prec	0.5265	Rec	0.7346	F1	0.5980
0.40	IoU	0.4767	Dice	0.6028	Prec	0.5426	Rec	0.7178	F1	0.6028
0.45	IoU	0.4813	Dice	0.6065	Prec	0.5584	Rec	0.7008	F1	0.6065
0.50	IoU	0.4848	Dice	0.6091	Prec	0.5740	Rec	0.6836	F1	0.6091
0.55	IoU	0.4870	Dice	0.6105	Prec	0.5877	Rec	0.6659	F1	0.6105
0.60	IoU	0.4873	Dice	0.6104	Prec	0.6028	Rec	0.6474	F1	0.6104
0.65	IoU	0.4858	Dice	0.6086	Prec	0.6175	Rec	0.6277	F1	0.6086

□ Best threshold: 0.55 → Dice 0.6105

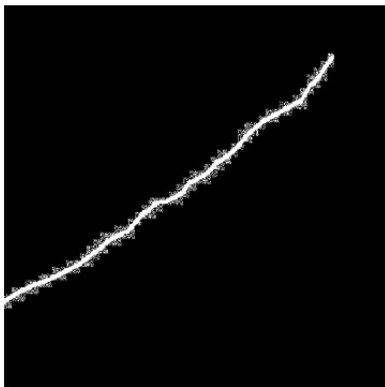
□ FINAL RESULTS (Flip TTA + post-processing):

IoU	:	0.4870
Dice	:	0.6105
Precision	:	0.5877
Recall	:	0.6659
F1 Score	:	0.6105

Input



Ground Truth



Prediction



Input



Ground Truth



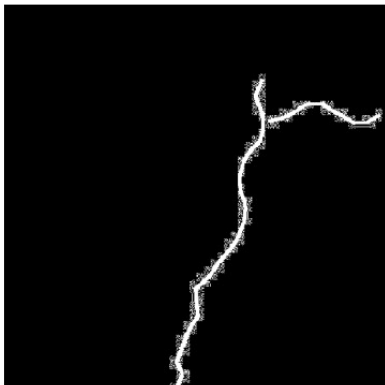
Prediction



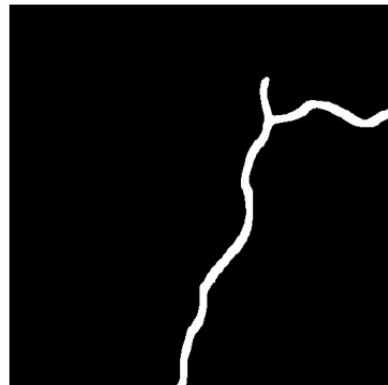
Input



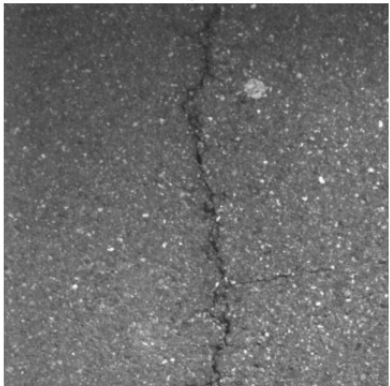
Ground Truth



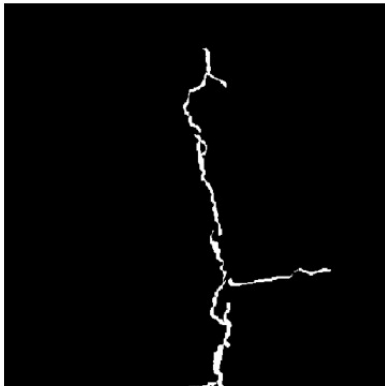
Prediction



Input



Ground Truth



Prediction

